

GitHub, estrutura do projeto e como clonar

BuscaML API — repositório `github.com/murilosouzza/BuscaML_API` (privado)

Documento gerado em 08/09/2026

1. O que foi feito no versionamento

Antes, a pasta do projeto estava “solta” dentro de outro repositório Git e não era versionada. Passos executados:

1. `git init -b main` dentro de `BuscaML_API/` — o projeto virou um repositório próprio.
2. Criado o `.gitignore` (ver seção 4).
3. Primeiro commit: `30559ca` — “Sprint 3: integração Mercado Livre”.
4. Repositório criado no GitHub (vazio, privado) e conectado: `git remote add origin ...`.
5. `git push -u origin main`.
6. Segundo commit: `ded76b1` — “Integração front + back” — e novo push.

Commit	Descrição
<code>30559ca</code>	Camada de dados, cliente do Mercado Livre, BuscaService (cache + fallback), serviço em segundo plano, controllers, autenticação OAuth e cliente fake.
<code>ded76b1</code>	Front movido para <code>wwwroot</code> , back passa a servir o site, CORS, <code>fetchProducts()</code> real, seed inicial.

2. Estrutura atual

```
BuscaML_API/                                <-- repositório Git (raiz)
|
|-- Controllers/                             back (C#)
|-- Services/                               back (C#)
|-- Models/                                 back (C#)
|-- Data/                                   back (C#)
|-- DTOs/                                  back (C#)
|-- Database/schema.sql                     back (referencia do banco)
|-- Program.cs                             back (ponto de partida)
|-- appsettings.json                       back (configuracao)
|
|-- wwwroot/                               FRONT (dentro do back)
|   |-- home.html
|   |-- busca.html
|   |-- app.js
|   |-- styles.css
|   +-- icones/
|
|-- .gitignore
+-- BuscaML_API.csproj
```

Não são mais duas pastas “front” e “back” separadas. O front virou uma subpasta do back (`wwwroot/`). É um projeto só, um repositório só.

3. Por que o front está dentro de `wwwroot`

`wwwroot` é o nome que o ASP.NET Core reconhece por convenção: tudo que está lá é servido como site, sem configuração extra. Colocando o front ali, ganhamos:

Vantagem	Explicação
Um comando só	<code>dotnet run</code> sobe o site e a API juntos, em <code>http://localhost:5102</code> .
Sem problema de CORS	Site e API na mesma origem (mesmo endereço). O navegador não bloqueia as chamadas.
Uma entrega só	Um repositório no GitHub, não dois. Mais simples de clonar e avaliar.
URLs relativas	O <code>app.js</code> chama <code>/api/buscar</code> sem endereço fixo — funciona local e em produção sem alterar código.

Não é irreversível: se um dia o front crescer a ponto de precisar de um projeto próprio (por exemplo, migrar para React/Vite), dá para separar em dois repositórios. Os arquivos continuam os mesmos.

4. O `.gitignore`

```
bin/           # resultado da compilacao
obj/           # arquivos temporarios de build
.vs/          # config do Visual Studio
*.user        # preferencias pessoais do VS
*.db *.db-shm *.db-wal # o banco SQLite local
appsettings.*.local.json # configuracoes locais
```

Esses itens **não vão para o GitHub** de propósito — são gerados na máquina de cada pessoa ou contêm dados locais.

5. Como clonar e rodar

```
git clone https://github.com/murilosouzza/BuscaML_API.git
cd BuscaML_API
dotnet run
```

Depois, abrir no navegador: `http://localhost:5102/`

Pronto. O site aparece já com produtos (dados fake), a busca funciona, a API responde.

Pré-requisito

Ter o **.NET 10 SDK** instalado. Sem ele o `dotnet run` não funciona.

O que se resolve sozinho no primeiro `dotnet run`

Item	Por que não está no repositório	O que acontece
buscaml.db (banco)	Ignorado no <code>.gitignore</code>	É criado automaticamente, com um seed de buscas iniciais.
bin/, obj/	Lixo de compilação	O <code>dotnet run</code> regenera.
Credenciais do Mercado Livre	São segredo — ficam no perfil do usuário, não no código	O projeto roda com o cliente fake . Para usar a API real, cada pessoa configura as próprias credenciais.

6. Modo fake x modo real

Por padrão, em desenvolvimento, o projeto usa o `FakeMercadoLivreClient` : 4 produtos fixos, sem internet. Isso permite clonar e rodar sem nenhuma configuração.

Para ligar o Mercado Livre real (quando houver as credenciais do app):

```
dotnet user-secrets set "MercadoLivre:ClientId" "SEU_ID"
dotnet user-secrets set "MercadoLivre:ClientSecret" "SEU_SECRET"
# e em appsettings.Development.json trocar "UseFake": true -> false
```

7. Repositório privado

O repositório foi criado como **privado** (cadeado no GitHub) — só o dono e convidados veem. Para compartilhar com um professor: `Settings > Collaborators > Add people`, ou tornar público em `Settings > General > Change repository visibility`.

O cadeado que aparece nos arquivos **dentro do Visual Studio** é outra coisa: significa “arquivo já commitado, sem alterações pendentes”. É o estado normal e esperado.

8. Fluxo do dia a dia

```
git add -A
git commit -m "descricao do que mudou"
git push
```